

含算子 714、716、722），其在向量流水线上密集执行，随后核心 1 推进执行其原有任务 C1:T1/SG1；核心 0 上则在完成保留支路 Y 后，紧接着执行汇合任务 C0:T5/SG5（子图 5，对应 J）。在两组时间线中，执行资源对应行的空白仅表示该时段没有记录到流水线占用事件，不能据此主观判定其原因是等待外部内存、片上存储或其他硬件资源。

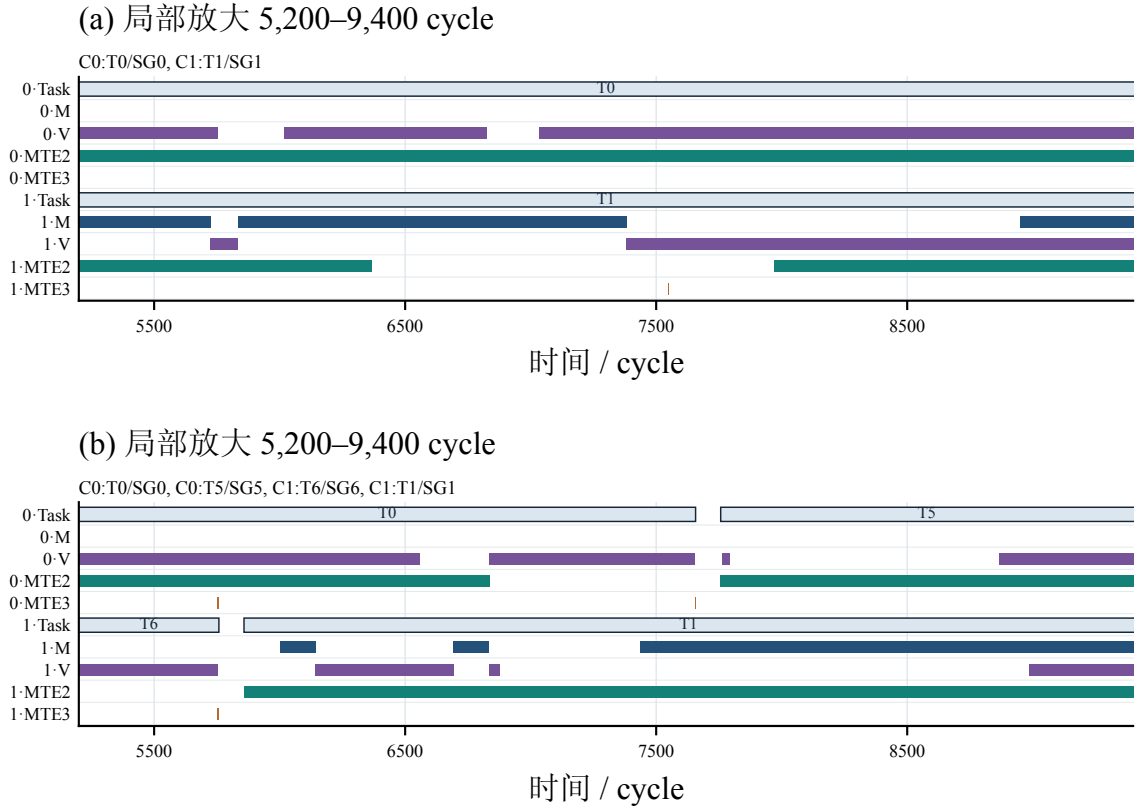


图 4-6 分支重分配前后的局部执行时序

图注：用例 case_026 在 5,200–9,400 cycles 窗口内的局部微架构流水线时序放大图（对应图 4-5 浅色区域）。展示核心 0 与核心 1 在该时段内的任务切换与各流水线操作事件分布。图中执行资源对应行的空白仅表示该时段未记录到流水线占用事件，不推定具体的外部等待原因。

该案例反映了多核心硬件调度中的典型权衡：通信量的局部上升并不必然导致整体性能劣化；当多核心并行推进缩短排队的效果超过数据搬运与同步延时，即可在官方模拟中获得全局完成时间的改善。

5 问题二：数据复用下的多核调度

5.1 问题分析与数据复用约束

在问题二的系统设定中，被分配到同一个计算核心的全部子图在底层编译时被合并为一个整体硬件执行任务（Task）。这一微架构语义明确了同核内的数据流动特征：在同一核心内部，先前执行的子图算子所生成的数据张量可以直接保留在该核心的片上私有缓存（L1 Buffer 与 Unified Buffer, UB）中，供后继同核算子直接读取复用，无需再经由外部主

存（DDR）进行持久化中转；而跨核心数据传递则依然必须经由外部主存通过跨核通信链路进行中转。

若数据生成算子与数据消费算子被分配至不同的计算核心，则数据依然必须通过跨核心通信链路传递：由源核心执行 `COPY_OUT` 将数据写入 DDR，再由目标核心执行 `COPY_IN` 将数据读入本地缓存。在硬件时序上，目标核心的读取操作必须在源核心写出操作彻底完成 500 cycles 之后方可启动。在此执行模型下，即使在方案提交文件（`node_to_subgraph`）中将每个计算算子单独划分为一个子图，也不会在硬件底层为每个算子生成孤立的物理 Task，子图序列仅用于指示底层编译器在各核心内部排定算子的先后执行次序。

同核心任务合并加深了算子分配与调度顺序之间的耦合关联，并引出了新的微架构存储权衡：

1. **缓存复用收益：**当某一数据张量的生成算子与全部消费算子均被分配至同一计算核心、且不存在其他跨核消费者时，可免去该张量对应的跨核 DDR 读写搬运与 500 cycles 的跨核同步等待；若仍有部分消费者位于远端核心或存在外部主存写出，则相应的边界传输依然发生；
2. **片上存储驻留压力与溢出代价：**若过度追求局部复用而将算子集中，或者消费同一张量的多个算子在执行时序上相距过远，该中间张量就必须在片上私有缓存中维持长时间的驻留生命周期。当片上缓存容量不足以容纳活跃数据峰值时，官方编译系统将在其内部流程的第二阶段（Step 2）强制插入溢出换出与换入操作（spill），从而在执行阶段产生意料之外的二次 DDR 读写开销；
3. **空间复用与时间并行的冲突：**将数据消费算子分散至其他核心虽会引入边界数据搬运，但能有效利用多核计算资源展开并行计算，并释放本地片上存储压力。

因此，私有缓存容量限制与 Step 2 溢出（spill）深刻影响着系统的实际数据搬运总量。多核心调度算法必须在控制跨核数据流动的同时，防止片上工作量与存储过度聚集。

图 5-1 采用一个包含 9 个物理张量的手算结构用例，展示了单核心排队执行时片上私有缓存占用的动态变化过程。需要指出的是，该图展示的是单核心在静态编译序列下的结构示意，横轴展示操作序号 $i = 0 \dots 7$ （其中 0 与 7 分别对应初始与全部释放的空占用边界；该序号为模仿官方编译第二阶段 Step 2 静态操作顺序的手算示例，展示顺序编译次序而非微架构物理并行执行周期 cycles），并非全量 100 张基准计算图的实测记录或真实的并行执行时间线。每个物理张量 t 在该核心的生命周期定义为闭区间 $[f(t), \ell(t)]$ ，分别对应在该静态编译序列中第一次与最后一次触碰该物理张量的操作序号（末次触碰包含计算读取或外存写出等搬运操作）；张量在末次触及步骤 $\ell(t)$ 期间仍持续驻留在片上私有缓存中，直至后继步骤启动时方被完全释放。具体而言，张量 T_6 的生命周期区间为 $[5, 6]$ ，张量 U_3 为 $[4, 6]$ 。在该手算用例中，容量曲线未超出配置上限属于基于静态步数累加的手算事实；由于本示例并未运行官方离散事件仿真，图中无官方 spill 事件记录且未绘制溢出事件。在单核心物理容量（L1 Buffer 容量为 512 KiB，Unified Buffer 容量为 128 KiB）约束下，若某步骤同时存在多项未释放的输入张量与新分配的输出张量，将产生瞬时存储占用峰值（图中示例在步骤 2 达到 L1 峰值 512 KiB，在步骤 4 达到 UB 峰值 72 KiB）。该示例为理解调整算子顺序能够规避潜在溢出换出提供了物理直观；本文主算法并未在此建立全局无溢出的精确容量调度保证。在实际求解中，算法采用分层控制机制：超图割目标 D_{pre2} 用于度量 Step 2 插入缓存溢出换入换出前重建方案的 COPY 总字节数（包括基础输入输出常数项与分核指派引起的项），而流水线工作量上限（caps）仅约束各核心在各计算流水线上的操作计算时长之和；这两项约束均不保证片上私有缓存不发生容量溢出（spill），亦不构成全局完成时间（Makespan）必定缩短的理论保证。

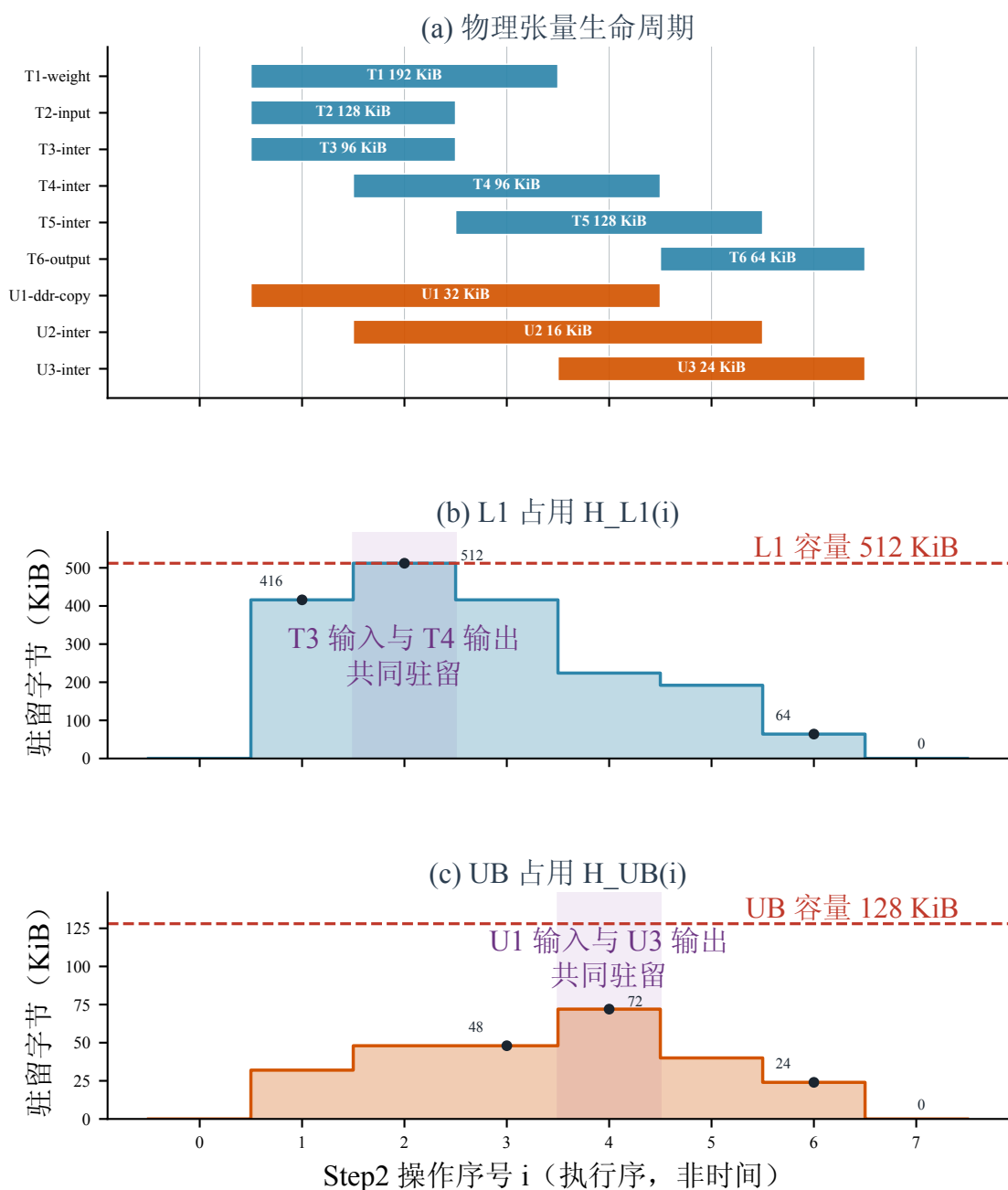


图 5-1 张量生命周期与片上存储占用

图注：单核心张量生命周期与片上私有缓存容量占用手算示例。上部展示 9 个物理张量在闭区间 $[f(t), \ell(t)]$ 内的驻留条带（如 $T_6 = [5, 6]$ 与 $U_3 = [4, 6]$ ）；中部与下部展示基于模仿 Step 2 静态编译操作序号下的 L1 Buffer（容量 512 KiB，步骤 2 达峰值 512 KiB）与 Unified Buffer（容量 128 KiB，步骤 4 达峰值 72 KiB）存储占用阶梯曲线，说明输入与输出张量在操作边界共同驻留对局部容量峰值的影响。图中无官方 spill 事件记录且未绘制溢出事件，横轴为静态操作序号而非微架构物理执行周期。

5.2 基于超图的复制量模型

同一个核心保留数据与在不同核心之间复制数据的区别见图 1-1。后续按这个区别计算边界复制量。

5.2.1 超图表示与适用条件

在普通图模型中，传统图割算法将数据流动简化为边权切分，无法准确刻画“一对多”数据复用的真实开销。

考察一个典型的复用场景（图 5-2）：算子 u 生成大小为 8 B 的张量 t ，算子 v_1 与 v_2 均读取 t 。在硬件执行层面上，若将读取算子 v_1 与 v_2 分配至同一个远端核心，该核心只需从 DDR 读入一次 8 B 数据即可供两个算子共同使用（对应源核心一次 COPY_OUT 与目标核心一次 COPY_IN，共产生 16 B 边界流量）；若仅将 v_1 移回 u 所在核心，由于 v_2 仍驻留在远端，该远端核心依然需要完整搬入该张量，边界复制量依然维持 16 B；只有当 v_1 与 v_2 均被移回 u 所在核心时，跨核数据传递需求才彻底消失，相关边界复制量降为 0 B。在不同的核心分配下，该张量引发的边界复制量分别为 16 B、16 B 与 0 B。这种非线性特征表明，边的割断无法等价于数据复制。

为精确刻画多算子共同使用同一数据的共享边界，本文引入**超图**（hypergraph）模型 $\mathcal{H} = (V, \mathcal{E})$ 。超图由算子顶点集 V 与**超边**（hyperedge）集合 $\mathcal{E}$ 构成，每条超边连接任意非空的顶点子集（允许包含单个顶点）[13]。在并行计算与数据划分领域，利用超图连接度模型刻画多处理器间的通信开销是经典的方法论基础[14]。超图仅表示算子间的数据关联关系，不指示计算先后顺序；原计算图的有向拓扑依赖另行独立维护。

模型定义与适用条件 设输入计算图的所有操作集合为 O 。定义待调度的操作集合为

$$V = \{u \in O \mid \text{op}(u) \notin \{\text{COPY_IN}, \text{COPY_OUT}\}\}, \quad (5-1)$$

即排除跨边界核外搬运操作 COPY_IN 与 COPY_OUT 之后的全部待调度操作集合（其中计算算子与子图内部搬运操作均可能包含在 V 内）。

超图模型严格适用于如下计算图领域：

1. 顶点集 V 完整包含上述除核外搬运操作外的全部原始待调度操作；
2. 在原始计算图 O 中，每个物理张量至多由一个原始操作生成（即 $|\text{producers}(t)| \leq 1$ ）；严格拒绝逻辑张量别名（logical_tid），张量大小 b_t 为非负整数；
3. 直接操作间依赖边（direct op-op edges）亦纳入模型统一处理。

5.2.2 超边与复制代价

对于计算图中的每个物理张量 t ，定义其在待调度操作集合 V 内的生成操作集合与消费操作集合为：

$$p_V(t) = \text{producers}(t) \cap V, \quad c_V(t) = \text{consumers}(t) \cap V. \quad (5-2)$$

由于原始图中张量生成者至多一个，故集合 $p_V(t)$ 要么为空集，要么包含唯一的生成操作。

超边集 $\mathcal{E}$ 与固定常数项 C 按如下规则精确构建：

1. **无关联张量略去**：若 $p_V(t) = \emptyset$ 且 $c_V(t) = \emptyset$ ，该张量在待调度操作集 V 内既无生成者亦无消费者（例如纯粹在外部搬运操作之间流转的张量），在超图模型中直接略去，不建立对应超边；

2. **模型外部输入张量**: 若 $p_V(t) = \emptyset$ 且 $c_V(t) \neq \emptyset$ (例如由外部 DDR 读入或由图内 COPY_IN 载入但生成操作不在 V 中的张量), 该张量在待调度操作层面上构成外部输入。其首次载入的基础开销 b_t 计入全局常数项 C ; 其超边引脚集合仅包含读取该输入的待调度操作子集 $c_V(t)$ 。每增加一个需要该输入的核心, 需额外从 DDR 搬入 b_t 字节, 故超边权重赋值为 $w_e = b_t$;
3. **模型内部生产张量**: 若 $p_V(t) \neq \emptyset$ (记唯一的生成操作为 $u \in V$), 其超边引脚集合包含生成操作及全部消费操作 $c_V(t) \cup \{u\}$ 。依官方冻结评测编译语义, 跨核心访问时涉及源核心写出外存与目标核心读入本地的一对操作, 故超边权重赋值为 $w_e = 2b_t$ (硬件常数严格依据源码, 不作未经证实的广播假设); 若该张量同时被外部 COPY_OUT 操作消费, 或者在计算图内无后继待调度消费操作 ($c_V(t) = \emptyset$), 表明其结果必须写出至外部主存, 其写出外存的固定基础开销 b_t 累加至常数项 C ;
4. **直接操作间依赖边**: 对原始图中源节点与目标节点均属于 V 的直接操作依赖边 $e = (u, v)$, 构建引脚集合为 $\{u, v\}$ 、权重为 $w_e = 2 \times \text{data_size}(e)$ 的超边;
5. 引脚集合为空或权重为零的超边被直接丢弃, 不加入超边集 $\mathcal{E}$ 。

换入换出前的原始复制字节数 给定待调度操作到计算核心的指派映射 $a : V \rightarrow \{0, 1, \dots, K-1\}$, 定义 $\lambda_e(a)$ 为超边 e 中全部操作所跨越的不同核心数量 (无量纲整数)。在官方评测程序第二阶段 (Step 2) 插入缓存溢出换入换出之前, 方案的**预 Step 2 原始复制字节数** (pre-Step2 original COPY bytes) 精确满足:

$$D_{\text{pre2}}(a) = C + \sum_{e \in \mathcal{E}} w_e (\lambda_e(a) - 1). \quad (5-3)$$

其中, D_{pre2} 、 C 与 w_e 的单位均为字节 (B)。当一条超边的所有相关操作均指派给同一核心时, $\lambda_e(a) = 1$, 对求和式的贡献为 0; 每向一个新核心延伸, 均线性增加 w_e 字节。该指标精确度量了 Step 2 插入缓存溢出换入换出前重建方案的 COPY 总字节数 (包括基础输入输出常数项 C 和分核指派引起的跨核边界项), 并不等同于官方离散仿真中扣除原始图 COPY 量后的额外 DDR 搬运量 (added_copy_bytes), 亦不代表 Makespan。

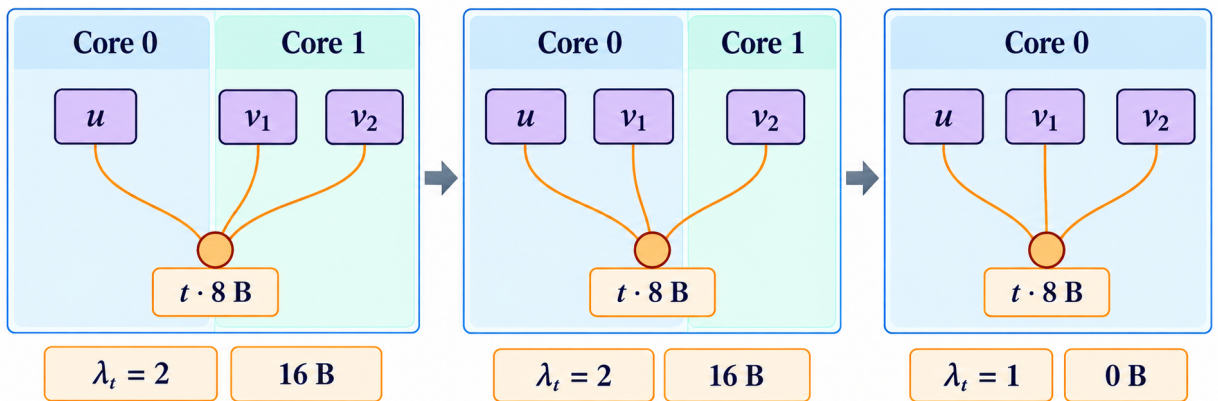


图 5-2 张量相关算子的分核与复制量

图注: 同一张量的计算操作分配。图中 u 生成 t , v_1, v_2 读取 t ; λ_t 表示这三个操作所在的不同核心数量。16 B、16 B、0 B 仅统计该张量的边界复制, 线条只表示集合关联。

5.2.3 链分组移动的代价增量

当算法将一组算子 $A \subset V$ 从源核心 s 整体迁移至目标核心 t ($s \neq t$) 时, 要求 A 中的所有算子在迁移前必须处于同一源核心 s (不支持跨源核心的混合分组移动)。

对于任意与 A 相交的超边 e , 令 $n_{e,s}$ 与 $n_{e,t}$ 分别为迁移前超边 e 在源核心与目标核心上的引脚数量, 令 $m_e = |e \cap A|$ 为超边 e 中被迁移的引脚数。超边 e 的复制字节变化量 Δ_e 严格满足:

$$\Delta_e = w_e (\mathbf{1}\{n_{e,t} = 0\} - \mathbf{1}\{n_{e,s} = m_e\}). \quad (5-4)$$

其中 $\mathbf{1}\{\cdot\}$ 为指示函数。若目标核心原先未包含该超边算子, 则新增一个跨核副本 (增加 w_e); 若源核心所包含的该超边算子被全部移走, 则彻底消除源核心的副本需求 (减少 w_e)。总变化量为所有相交超边 Δ_e 的代数和。在连续执行多次分组调整时, 算法只需增量维护各超边在各核心的引脚计数 $n_{e,k}$ 。

5.3 基于局部最小割的模型求解

5.3.1 基于空闲区间的初始分配

为了在调度构建阶段兼顾计算流水线的利用率, 算法设计了基于空闲时间区间 (gap intervals) 的拓扑编排机制。该机制实时维护各计算核心在输入计算图中实际出现的各类执行流水线 (如 PIPE_M、PIPE_V 等) 上已排定算子的占用时间区间集合。

在处理计算链时, 算法沿链内依赖拓扑顺序依次为各算子寻找可行空闲区间。设待排定算子 v 的计算时长为 $d_v = \max(1, \text{cycles}_v)$, 其在当前部分调度下的估计最早数据就绪时刻为 r_v 。记 $\mathcal{I}_{k,p_v}$ 为核心 k 上对应流水线 p_v 已占用时间区间的并集。算法在满足数据依赖约束的前提下, 在时间轴上寻找最早满足互不重叠条件的空闲时间窗口, 选取最早的可行启动时刻 s_v :

$$s_v \geq r_v, \quad [s_v, s_v + d_v) \cap \mathcal{I}_{k,p_v} = \emptyset. \quad (5-5)$$

时间区间采用左闭右开表示, 允许后继算子在前序算子完成的时钟周期无缝衔接。在多支路汇聚节点处, 为了避免局部贪心导致单侧支路过度滞后, 算法在汇合算子两侧至多联合枚举 K^2 种核心指派组合, 综合评估不同核心搭配下的完成时刻。

需要强调的是, 上述空闲区间计算仅为引导候选方案构造的启发式时间代理 (proxy heuristic), 其并未模拟底层 DDR 访问仲裁、总线带宽争用以及复杂的片上存储空间动态分配。因此, 估计时刻 s_v 既不构成全局 Makespan 的数学上界, 亦不构成理论下界。方案的物理执行时刻最终由官方离散事件仿真器推进确定。

5.3.2 局部超图割与辅助流网络

为了优化上述超图连接度目标, 算法在空闲区间构造方案的基础上, 选取两个核心 a, b 以及最多 16 条前后相连的计算链 (chain, 总数 $w \leq 16$) 作为局部调整区域。所选区域之外的全部算子核心分配保持锁定, 区域内的各条计算链在核心 a 与 b 之间决策归属。

辅助网络构造与等价性 在固定范围外算子后, 最小化二核心之间的超图连接度可通过构建有向辅助网络转化为源汇最小割 (minimum source-target cut) 问题。利用有向辅助网络将特定超边切分代价转化为源汇割模型在超图割文献中已有理论研究 [15]; 本实现针对固

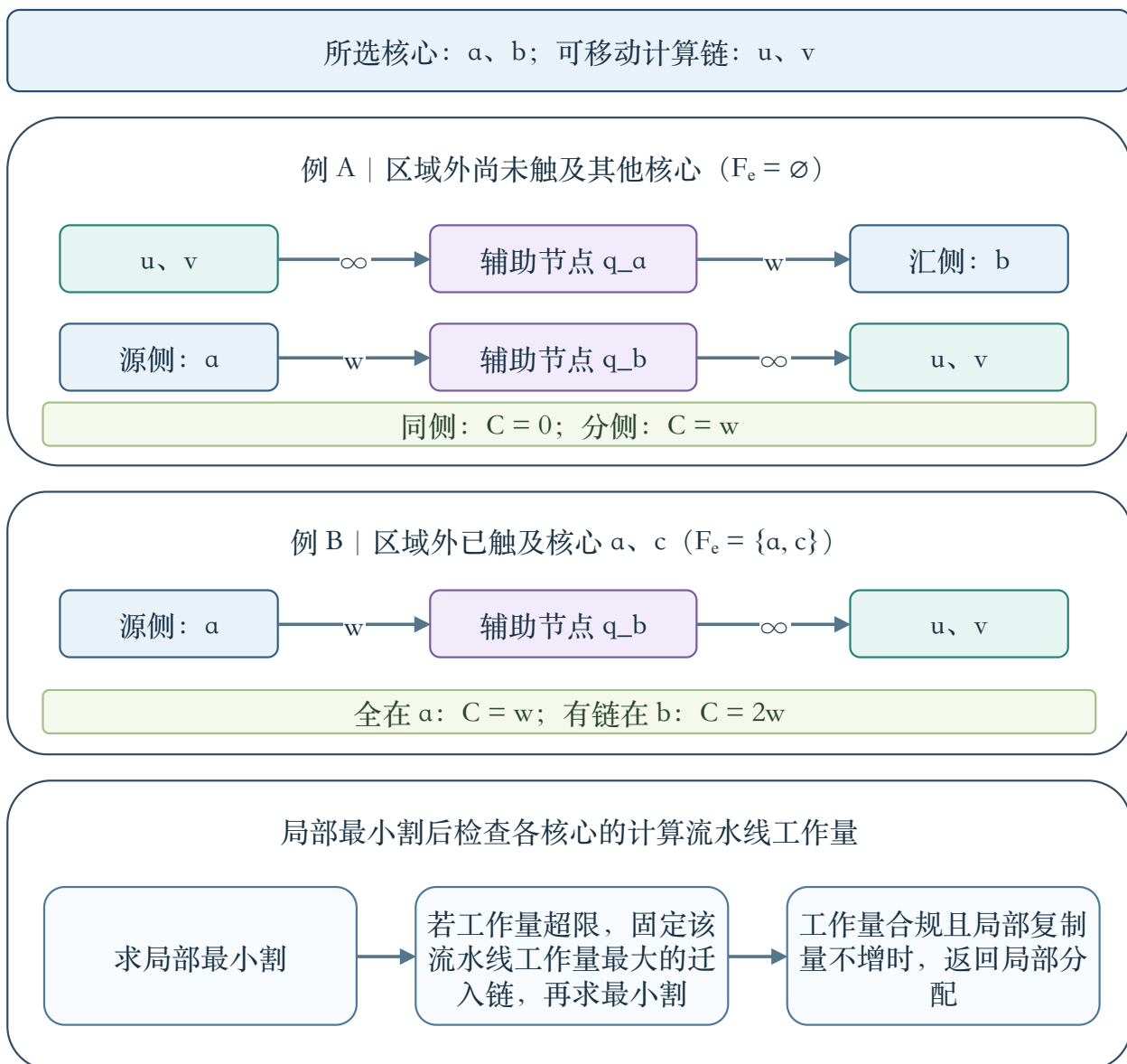


图 5-3 二核心超图割构造与工作量保护

图注：二核心局部超图割辅助网络构建与工作量保护流程。展示超边在固定核心集合为空集与非空集合时的有向辅助网络映射，以及结合流水线工作量上限保护的迭代锚定求解机制；图中为简化连线在上下两行重复绘制了一组可移动链 u, v （对应同一对决策单元而非四个独立节点），图内辅助节点 q_a, q_b 分别对应正文中的 aux_a, aux_b 。局部受保护割至多执行 $r + 1$ 次最大流求解（ $r \leq 16$ 为自由移动链数量），初始分配始终可行，与全局至多 3 次在线评测预算完全解耦。

定引脚与二核心指派给出具体的 OR 辅助网络映射与代数证明，该构造并不推广至任意次模分割函数，亦不推出带容量约束的全局 Makespan 最优。

设源点对应核心 a （割的源侧为分配至 a 的单元），汇点对应核心 b （割的汇侧为分配至 b 的单元）。对于每条超边 e ，其在局部自由可移动单元集合之外的固定引脚所归属的核心集合记为 F_e （ F_e 可能包含核心 a 、核心 b ，亦可能包含其他计算核心，且可能为空集）。

算法无条件累加所有超边的固定项至基准常数：

$$\text{offset} = \sum_{e \in \mathcal{E}} w_e (|F_e| - 1). \quad (5-6)$$

特别地，当某条超边在调整区域外无任何固定引脚时， $F_e = \emptyset$ ，其贡献项为 $-w_e$ 。
网络弧权与辅助节点按如下规则构造：

1. 有限弧权和与无穷容量设定：

$$\text{finite_sum} = \sum_{e \in \mathcal{E}} w_e (\mathbf{1}\{a \notin F_e\} + \mathbf{1}\{b \notin F_e\}), \quad M_\infty = \text{finite_sum} + 1. \quad (5-7)$$

2. **核心 a 覆盖代价辅助节点 aux_a** ：若 $a \notin F_e$ ，建立辅助节点 aux_a 。从超边 e 内的各个自由计算链节点向 aux_a 连接容量为 M_∞ 的有向弧，并从 aux_a 向汇点连接容量为 w_e 的有向弧。若该超边有任意一条链被分配给核心 a （落入源侧集合 S ），最小割必须切断容量为 w_e 的弧；
3. **核心 b 覆盖代价辅助节点 aux_b** ：若 $b \notin F_e$ ，建立辅助节点 aux_b 。从源点向 aux_b 连接容量为 w_e 的有向弧，并从 aux_b 向该超边的各个自由计算链节点连接容量为 M_∞ 的有向弧。若有任意一条链被分配给核心 b （落入汇侧集合 T ），最小割必须切断容量为 w_e 的弧；
4. **锚定弧 (anchors)**：若某计算链单元 u 被强制锁定在核心 a ，则添加容量为 M_∞ 的源点指向 u 的弧；若锁定在核心 b ，则添加 u 指向汇点的 M_∞ 弧。

该辅助网络求得的源汇最大流数值记为 $f_{\max}$ ，局部超图连接度严格满足恒等关系：

$$\text{Cost} = \text{offset} + f_{\max}. \quad (5-8)$$

在具体实现中，无穷大容量实际取为严格大于全部有限弧权之和的常数 $M_\infty = \text{finite_sum} + 1$ ，确保割弧仅落在对应核心覆盖代价的有限容量弧上。

图 5-3 详细给出了两种典型固定边界下的辅助网络构造与代价核算示例：

1. **例 A（固定核心集合为空， $F_e = \emptyset$ ）**：超边连接局部两条可移动链 u, v ，超边权重为 w 。基准偏移为 $\text{offset} = -w$ 。当给定 u, v 同归属于某一核心时（例如均分在核心 a 或均分在核心 b ），辅助节点在较优侧时的对应割容量为 $c_{\text{cut}} = w$ ，加上偏移后该分配下的局部复制代价为 $\text{Cost} = -w + w = 0$ ；若 u, v 分跨两个核心，两侧辅助节点的容量边均被割断，对应割容量为 $c_{\text{cut}} = 2w$ ，局部复制代价为 $\text{Cost} = -w + 2w = w$ ；
2. **例 B（已固定触及核心 a 与另一核心 c ， $F_e = \{a, c\}$ ）**：由于核心 $a \in F_e$ ，系统不再为核心 a 建立容量边，仅当有链移入新增核心 b 时才产生新增复制，基准偏移为 $\text{offset} = (|F_e| - 1)w = +w$ 。若给定链全部保留在核心 a ，对应割容量为 $c_{\text{cut}} = 0$ ，局部代价为 $\text{Cost} = w + 0 = w$ ；若有链被分配至核心 b ，割断指向核心 b 的容量边（权重 w ），对应割容量为 $c_{\text{cut}} = w$ ，局部代价为 $\text{Cost} = w + w = 2w$ 。对任意给定的局部链分配，辅助网络在最优辅助节点放置下的割容量 c_{cut} 加上偏移均严格等于超图连接代价；而在网络未加额外锚定时求得的最小割容量（即最大流值 $f_{\max}$ ），则对应无约束局部最优分配，满足总体最优成本 $\text{Cost}^* = \text{offset} + f_{\max}$ 。

需要强调的是，辅助网络中的局部超图割仅针对预 Step 2 原始复制字节数 D_{pre2} 进行直接建模，并以 cycles 为单位对各计算流水线施加工作量上限保护，它并不保证片上私有缓存不发生换出（spill 为 0）。同时，load_guarded_cut 本身返回的是满足工作量上

限且局部复制代价不增加（非严格增， $\text{Cost} \leq \text{Cost}_0$ ）的分配；而在外层细化流程 `gap_hyperrefine` 中，触发重排生成完整 P_2 候选的门槛则严格要求全图 $D_{\text{pre}2}$ 产生严格下降（ $\text{after} < \text{before}$ ）。此外，局部割容量的下降并不证明完整多核调度方案的 **Makespan** 必定缩短，各候选方案的综合性能仍须依赖官方离散事件仿真器评价。

5.3.3 流水线工作量约束与回退

单纯追求复制字节最小化可能导致最小割将绝大部分计算集中指派至单一核心，诱发严重的计算延迟倾斜。为此，算法引入了受工作量保护的闭环最小割算法（`load_guarded_cut`）。

算法按传入的各流水线工作量上限（`caps`）进行检查（初始分配必须完全满足上限）。若求解最小割后的分配导致某核心在某流水线上的总工作量超出了预设上限，算法取首个超限的核心与流水线，在导致超额的迁入链中选出在该流水线上耗时最长的一条链（同工作量按单元编号升序破平），将其锁定回初始分配状态，并重新求解辅助网络的最小割。具体单次局部求解逻辑见算法 5-1。

算法 5-1 受工作量保护的二核心局部超图割算法

```

1 # 输入：决策链 order、超图边 rows、核心 (a, b)、初始分配 initial、工作
  ↪ 量 work 与上限 caps
2 # 输出：局部分配 labels、复制成本 cost（初始分配满足  $\text{loads}(\text{initial}) \leq$ 
  ↪  $\text{caps}$ ）
3 def load_guarded_cut(order, rows, a, b, initial, work, outside, caps
  ↪ ):
4     cost0 = connectivity_cost(initial, rows)
5     anchored = {} # 记录锁回初始核心的链集合（至多迭代  $|\text{order}| + 1$ 
  ↪ 次）
6     for _ in range(len(order) + 1):
7         labels, cost = binary_hypercut(order, rows, a, b, anchored)
  ↪ # 求解最大流与割成本
8         loads = compute_loads(labels, work, outside)
  ↪ # 统计各核心流水线工作量
9         violations = [(c, p) for c in (a, b) for p in sorted(caps[a
  ↪ ]))
10                        if loads[c][p] > caps[c][p]]
11     if not violations:
12         assert cost <= cost0 # 满足上限时保证局部复制成本不递增
13         return labels, cost
14     core, pipe = violations[0] # 取首个超限的核心与流水线
15     # 在迁入该核心且具有该流水线正工作量的流动链中，选耗时最长者（ID
  ↪ 破平）
16     movers = [u for u in order if labels[u] == core and initial[
  ↪ u] != core
17                  and work[u].get(pipe, 0) > 0]

```

算法 5-1 (续)

```
18     chosen = min(movers, key=lambda u: (-work[u][pipe], _key(u))  
    ↪ )  
19     anchored[chosen] = initial[chosen] # 锁定回初始核心重新求解割  
20     raise AssertionError("exceeded finite anchoring bound")
```

源码对应: `c66559a, binary_hypercut.py::load_guarded_cut`。输入校验与 `GuardedCutResult` 结构打包见源码第 198–240 行。算法在局部至多求解 $r+1$ 次最大流割 ($r \leq 16$)，保证局部预 Step 2 复制成本不增加且各流水线工作量不超过预设上限；该机制旨在防止计算倾斜，不保证片上私有缓存无换出 (spill 为 0)，亦不保证全局 Makespan 缩短。

命题 5.1 (局部复制量与工作量保护性质)。若初始分配满足流水线工作量上限，且每次锚定均将单元固定回其初始核心，则 `load_guarded_cut` 算法在至多 $r+1$ 次最大流求解内必终止，且终止时所得方案的局部预 Step 2 复制量 D_{pre2} 绝不大于初始分配，且各流水线工作量均不超过上限。

证明. 初始分配始终完全满足工作量上限约束。由于每次锚定均与初始分配一致，初始分配在每一次增加锚定约束后的辅助网络中始终构成一个合法的割。根据最小割的最优性，求得的最小割代价必不高于初始分配的代价值。由初始分配完全满足各流水线工作量上限且算子工作量非负，若某一核心在某流水线上的总工作量超过上限，则该核心在该流水线上的工作量必然严格大于初始值，从而必然存在自其他核心迁入该核心且在该流水线上具有正工作量的流动链；又因已锚定单元只能留在其初始分配的核心，不可能作为迁入者，故待选的迁入链中必存在尚未被锚定的流动单元。由于局部自由链的总数为 r ($r \leq 16$)，若每次迭代均发生工作量超限，每次至少有一条未锚定的流动链被永久锁定回原位置。在最坏情况下，经过至多 r 次锁定后，全部流动链均被重置回初始分配，此时工作量超限自然消除。因此，算法必在至多 $r+1$ 次求解内终止，且终止解的局部复制字节数不高于原方案。 $\square$

需要明确指出的是，命题 5.1 证明的仅是在给定的局部二核心区域及锚定启发过程下预 Step 2 复制量不增加且流水线工作量不超限。局部受保护割最多执行 $r+1$ 次最大流求解 ($r \leq 16$ 为自由移动链数量)，与全局至多 3 次在线离散仿真评测预算完全解耦。流水线工作量上限是通过迭代式的启发式锚定进行防护，而非单次带约束规划的全局最优解；该机制并不保证片上私有缓存不发生换出 (spill 为 0)，并不保证全局 Makespan 一定缩短，亦非全图范围的多核全局最优解。算法仍须依赖最终的离散事件仿真来进行综合评价。

5.3.4 候选方案生成与择优

图 5-4 完整呈现了问题二候选方案的构造分支、去重与受限评测决策流：

1. **基础方案 P0 保底：**P0 由自适应装箱生成并始终保留，当后继候选不支持、构造异常或评测不可用时立即作为安全回退方案输出；
2. **空闲区间方案 P1：**尝试构造 P1，即使 P1 与 P0 方案相同，算法仍以原始 P1 为基准启动局部超图调整；若 P1 构造失败，则直接返回保底方案 P0；
3. **局部改进方案 P2 的门槛控制：**以 P1 为基础尝试局部超图调整，仅当调整前后的预 Step 2 原始复制字节数 (pre-Step 2 original COPY bytes) 均为有效非负整数且满足严格下降 ($\Delta D_{\text{pre2}} < 0$) 时，方调用重定时模块尝试生成完整的 P2 方案 (hypergap)；若局部调整不支持、重排失败或字节数未严格减少，则跳过 P2 构造，保留已有方案；若发生非预期错误或字节数证据无效，则直接返回 P0；

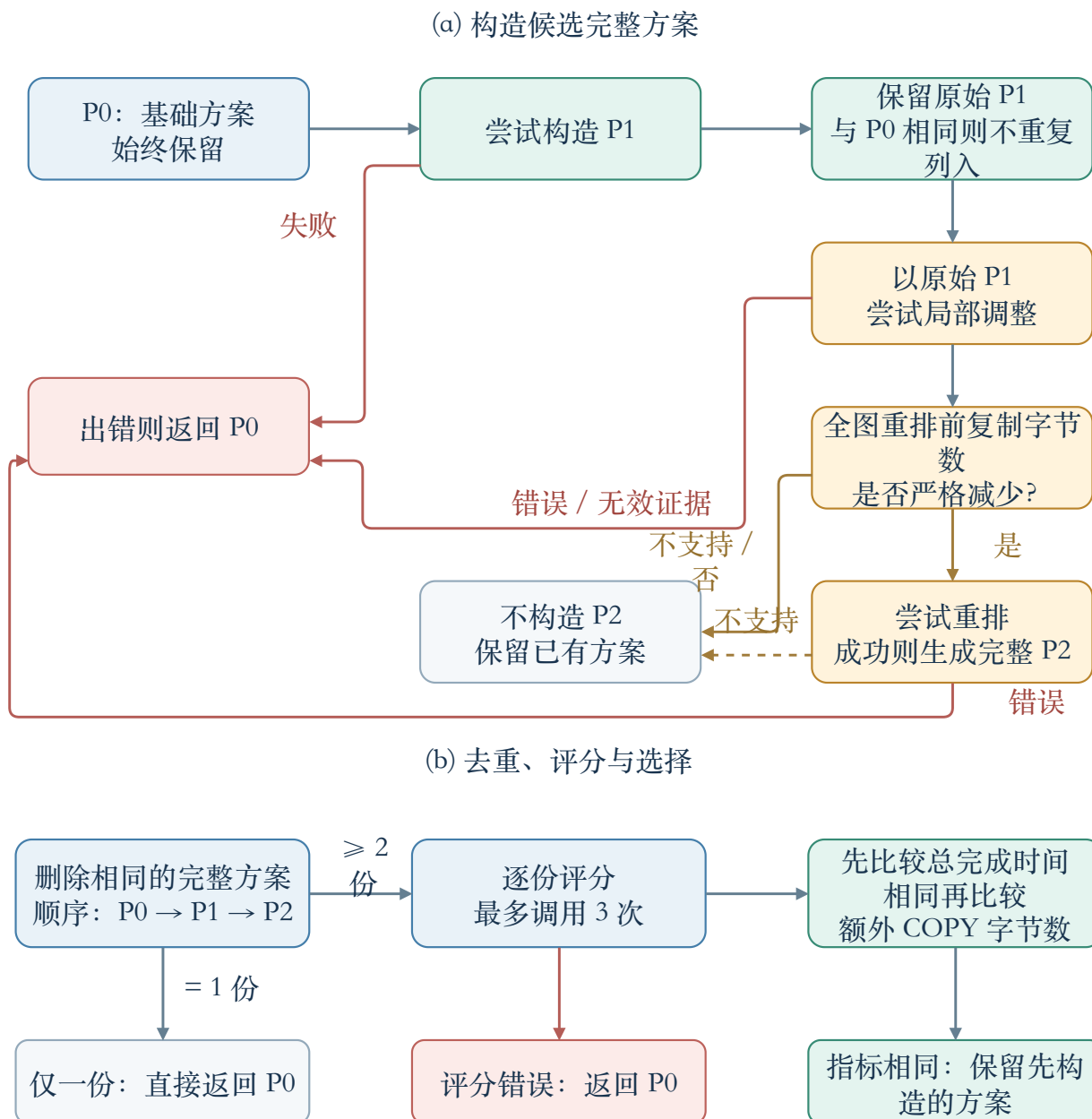


图 5-4 问题二候选构造与选优流程

图注：问题二三种候选方案的构造分支与评测选优决策流。展示基础方案 P0、空闲区间方案 P1 与局部超图调整方案 P2 的构造分支条件、按构造顺序对象值比对去重机制以及至多 3 次在线评分的字典序择优规则。

4. **按序去重与受限评测：**完整方案按 P0 → P1 → P2 的构造顺序，通过 Python 方案对象值比对进行去重（非哈希或序列化），避免对相同方案重复评测。若去重后仅有 1 份有效方案（即仅剩 P0），直接输出 P0 而不发起任何在线评测；若存在 2 份或 3 份互不相同的完整方案，按构造顺序逐份向在线评分接口提交请求（总调用次数不超过 3 次）；
5. **字典序择优与安全回退：**评分接口返回原生评测结果，算法依据目标向量 (M_2 , added_copy_bytes) 按完成时间优先、额外 COPY 字节数次之的严格字典序择优；指标相同时保留先构造的方案。若在线评分抛出异常或返回结果无效，控制器捕获后安全回退并输出保底方案 P0。

算法 5-2 问题二自适应超图细化与多核调度算法

```

1 # 输入: 原始计算图 graph、核心数 cores、硬件配置 config、在线评测接口
  ↪ oracle
2 # 输出: 选中的多核调度方案 winner_plan (P0 保底, 评测总数 ≤ 3 次)
3 def build(graph, cores, config, oracle):
4     baseline, _ = adaptive_budget.build(graph, cores, config) # 构
  ↪ 造 P0 保底方案
5     plans = [("baseline", baseline)]
6
7     # 构造空闲区间方案 P1 (结构不支持或异常回退 P0; 即使 P1 == P0 亦以此
  ↪ 细化)
8     gap = safe_build_gap(graph, cores, config) # 伪代码抽象: 异常捕
  ↪ 获与保底
9     if gap is None:
10         return baseline
11     if gap != baseline:
12         plans.append(("gap", gap))
13
14     # 局部超图优化: 显式区分跳过与致命异常 ( $\Delta D_{pre2} < 0$  判定与异常控制)
15     status, hyper = safe_refine_and_retime(graph, gap, config)
16     if status == "fatal":
17         return baseline # 非预期异常或证据无效直接回退保底
18     if status == "ok" and all(hyper != p for _, p in plans):
19         plans.append(("hypergap", hyper))
20
21     if len(plans) == 1:
22         return baseline # 仅剩唯一方案不发起在线评测
23
24     # 按构造顺序评测 (≤ 3 次), 按 (Makespan, added_copy_bytes) 严格
  ↪ 字典序择优
25     return select_best_by_lexicographic(oracle, plans, baseline)

```

源码对应: c66559a, adaptive_hypergap_guarded.py::build (其中 safe_build_gap、safe_refine_and_retime 与 select_best_by_lexicographic 为伪代码抽象)。safe_refine_and_retime 校验预 Step 2 原始复制字节数是否为有效非负整数, 仅在严格下降 ($0 \leq \text{after} < \text{before}$) 时调用重排; 结构不支持或重排跳过返回 ("skip", None), 遇到非预期异常或无效证据返回 ("fatal", None); select_best_by_lexicographic 规定任一次评测异常均全流程回退保底方案, 评测成功时按字典序择优, 平局保留先构造方案。

在上述决策流程中, 需要严格区分两个具有不同微架构语义的通信统计量:

1. **预 Step 2 原始复制字节数** (pre-Step 2 original COPY bytes, D_{pre2}): 是在候选构造阶段由超图连接度模型所度量的通信量, 反映 Step 2 插入缓存溢出换入换出前重建方案的

COPY 总字节数（包括固定输入输出项和分核指派引起项），作为决定是否调用重排模块尝试生成 P_2 的内部触发门槛，并不等同于最终完成时间的缩短；

2. **额外 COPY 字节数** (`added_copy_bytes`)：是在完整方案经官方离散事件仿真评测后，记录的调度 COPY 总量扣除原始图 COPY 量后的额外数据搬运量，作为多目标选优中的第二排序键。

评分函数仅在评测返回结果为字典映射 (`Mapping`)、状态为正常 (`status == 'ok'`) 且 `Makespan` 与额外 COPY 字节数均为有效非负整数时采纳该得分向量。全量实验中原生仿真器 E2 正常返回原生评测结果（未触发回退）属于实测数据事实，并非代码中的前置校验谓词。图中的“至多 3 次”指在线求解器内部的评分调用上限，独立的外部 E0 复评作为离线验证单独报告，不计入在线求解调用账。

5.3.5 时间复杂度分析

算法在求解过程中将各阶段计算开销解耦：

1. **图拓扑识别与链分解**：构建算子依赖拓扑并提取独立计算链；
2. **空闲区间扫描**：在候选构造过程中，算子插入需遍历维护的时间段集合，设累计扫描的时间段总数为 Q ；
3. **汇聚点配对检查**：在多支路汇合节点处枚举核心指派组合，涉及至多 K^2 种局部搭配与对应的排定开销；
4. **局部超图最小割与保护循环**：设共优化 R 个局部区域。第 j 个区域的辅助网络包含 n_j 个节点与 m_j 条弧，内部包含 $r_j \leq 16$ 条自由移动链。采用 Dinic 算法 [16] 单次求解最大流的最坏时间复杂度为 $O(n_j^2 m_j)$ 。在工作量保护循环下，每个区域至多求解 $r_j + 1$ 次最大流，局部割部分总时间界为 $\sum_{j=1}^R (r_j + 1) T_{\text{cut}}(n_j, m_j)$ ；
5. **离散事件评测代价**：在线评测限制在至多 $b \leq 3$ 次原生 E2 评测调用，总时间包含评测耗时项 $\sum_{j=1}^b T_j$ 。

受评测仿真内部事件推进开销制约，常数次评测限制并不构成全流程低多项式时间的代数证明，求解耗时的可控性根本上依赖于有限候选与轻量级启发式保护机制的结合。

5.4 结果与分析

5.4.1 全量用例的性能比较

本文算法在官方提供的 100 张全量基准计算图、涵盖 1 至 5 核心的全部 500 种测试组合上进行了完整的端到端求解与评测，所有方案均顺利通过官方合法性校验并正常推进完成。测试结果依据 `paper/manuscript-v1/data/all-results.csv` 以及固定评测数据资产 (`p2-audit.json.gz`，对应求解器 Commit c66559a) 进行了精确复核。

下表分别列出题目规定的官方基准平均加速比，以及使用算法输出方案计算的平均加速比。

如表中数据所示：

1. **多核加速比与单核比值解析**：在 2 至 5 核心场景下，官方主口径平均加速比从 2 核心的 2.3167 提升至 5 核心的 4.5498。在单核场景下，算法输出方案相比官方单核基线取得了 1.2061 的 B/M 比值（反映算法输出的单核方案 `Makespan` 优于官方基准，不代表完成时间单调下降 20.61%）。表中两行在单核场景下的分子 B 均严格采用官方单核基准 `Makespan`，差异源于分母方案不同（官方主口径行以官方基准方案自身为分母，比值为 1.0000；第二行以算法实际输出方案为分母，比值平均为 1.2061）。与前一固定完整版

表 5-1 问题二各核心数下的平均加速比与求解时间汇总

核心数量	1	2	3	4	5
官方口径平均加速比	1.0000	2.3167	3.2356	3.9711	4.5498
优化方案 $\text{mean}(B/M)$	1.2061	2.3167	3.2356	3.9711	4.5498
改善/持平/退化	59/41/0	64/36/0	54/46/0	49/51/0	42/58/0
求解时间均值 / s	2.613	3.863	4.251	4.498	4.921

本相比，全量 500 种测试组合中共有 268 个用例缩短了 Makespan，232 个用例持平，实现零退化（ $\text{degrade} = 0$ ）；全网格共有 69 张计算图在至少一个核心数下相比前一版本取得缩短。图 5-5 展示了各核心数下的加速比与指标分布，此处对比来自算法完整版本更替而非单一机制的隔离消融；

2. **通信流量与多核权衡：**全量 500 个测试组合的额外 DDR 搬运总量从基准版本的 3,043,860,416 字节上升至 4,159,738,560 字节（增幅为 36.66%）。在 500 个测试组合中：共有 55 个用例实现了执行时间与搬运量双重下降；199 个用例执行时间缩短但搬运量有所增加；14 个用例执行时间缩短而搬运量持平；其余 232 个用例两项指标均保持不变。该分布反映了算法以主目标（Makespan）优先的择优取向：在版本迭代中观察到了执行时间下降与额外搬运量增加相伴随的成对数值变动，此处仅客观记录前后版本对比的数据呈现，不作未经独立消融隔离的因果机制归因；
3. **评测调用账与计算效率：**在全量 500 次独立求解过程中，累计发起 1131 次在线 native E2 评测请求，全部成功返回原生结果，未触发任何向 E0 的异常回退。独立外部 E0 仿真复验亦全部顺利完成。采用最近秩次法（nearest-rank，取第 $[0.95n]$ 位），全量 500 个测试组合的求解程序墙钟时间均值为 4.029 秒，95% 分位数（第 475 位）为 18.841 秒，最大求解时间为 40.187 秒；独立的外部 E0 仿真耗时均值为 1.410 秒，95% 分位数（第 475 位）为 6.592 秒，最大值为 13.980 秒。单次最大求解耗时为 40.187 秒，低于赛题建议所述的 5~10 分钟。

图 5-6 进一步展示了四种不同固定算法在 100 张基准计算图上的多核平均加速比对比，数据汇总自固定提交 7b65452 记录的完整评测结果。对比中包含四种固定实现：张量分组构造（tensor）、空闲区间构造（gap）以及本文主算法（c66559a）在 1~5 核心上各有 100 张图的完整结果；容量保护方法 F1 仅具有 5 核心下的 100 图完整数据，因此以独立点展示，不推断全核曲线。全部 1600 行数据统一采用官方情况 A 单核基线作为分子 B ，除以对应方案完成时间 M 后逐图取平均。在单核心处，三条曲线的真实观测均值分别为 1.056973（tensor）、1.167562（gap）与 1.206139（主算法），各方法单核方案均优于官方基准，对应正文中算法输出方案的 $\text{mean}(B/M)$ 口径，并非官方口径的基线参照 1；灰色虚线仅作为理想线性加速参照。在 5 核心场景下，本文主算法达到 4.549757，张量分组为 3.896037，容量保护 F1 为 3.888209，空闲区间调整为 3.676575；右侧针对 5 核心点进行了放大展开。四组数据体现了不同固定算法路线的整体求解表现，不能解释为单一模块的隔离消融。

图 5-7 进一步展示了本文主算法（c66559a）在全量 500 个测试组合上的方案质量与求解耗时分布：1. **质量与耗时双对数分布：**子图 (a) 呈现了官方 Makespan（cycles）与完整求解墙钟时间（秒）的双对数散点分布，直观反映了算法在不同图规模与核心数下的耗时特征，未呈现无界膨胀趋势；2. **分核经验累积分布：**子图 (b) 给出了各核心数（每组 100 用例）求解耗时的经验累积分布函数（Empirical Cumulative Distribution Function, ECDF，度量求解耗时小于或等于横轴给定秒数的用例比例）。在统计指标上，全量 500 格的求解耗

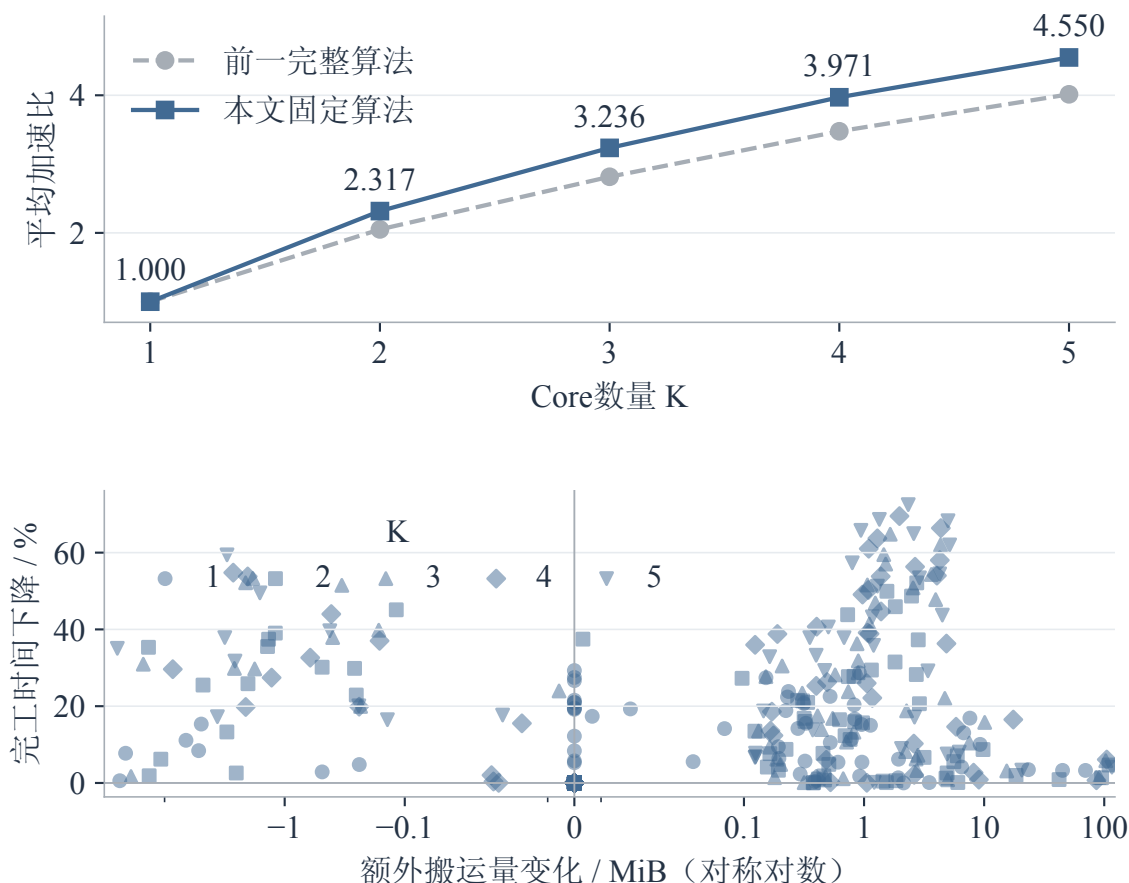


图 5-5 问题二算法版本的性能对比

图注：问题二的完整版本比较。上：按题目要求计算的平均加速比；下：500 种组合配对结果。部分时间改善伴随额外搬运增加，两项指标分别报告。

时中位数为 1.558479 秒，基于 $(n - 1)$ 线性插值法计算的 95% 分位数 (P95) 为 18.850035 秒，单次最大求解耗时为 40.186506 秒。需要说明的是，此处的线性插值 P95 数值与表 5-1 评测审计中采用的最近秩次法 (nearest-rank) P95 数值 18.841120 秒存在统计定义差异，两者均忠实反映实际耗时分布；3. 测量环境与边界口径：该批次在共享 macOS 环境（每分片 worker = 1，CPU 型号未记录）下运行，求解墙钟包含在线原生 E2 评测与文件持久化时间；外部独立 E0 复验耗时单列记录，不混入主求解耗时统计中，各用例单次求解耗时均低于赛题建议所述的 5~10 分钟。

5.4.2 搬运量与完成时间的权衡分析

为了检验“仅按片上容量和搬运量构造调度方案”是否存在固有缺陷，前期研发实验针对另一种固定构造策略（代码代号 C04）对六个预先指定的 5 核心用例进行了对比评测。

在该对照实验中，C04 生成的六份方案在格式与时序上完全合法，且经官方 Step 2 编译确认未插入任何由于容量超限导致的缓存换入换出 (spill 为 0)。然而，其最终 Makespan 全面劣于本文主算法，对比数据如下表所示：

尤其在 case_019、case_039 与 case_075 中，C04 方案的边界 COPY 字节数低于本文主算法，但其执行完成时间却出现了大幅退化（在 case_019 中变慢达 75.50%）。

图 5-8 与图 5-9 分别给出了 case_019（5 核心场景）在官方离散事件仿真下的全景与局部时序对比。数据显示，C04 方案的边界 COPY 字节数低于主算法（额外搬运量减少 73.20%），且无片上容量不足引发的 Step 2 缓存换入换出 (spill 为 0)，但其最终执行完成

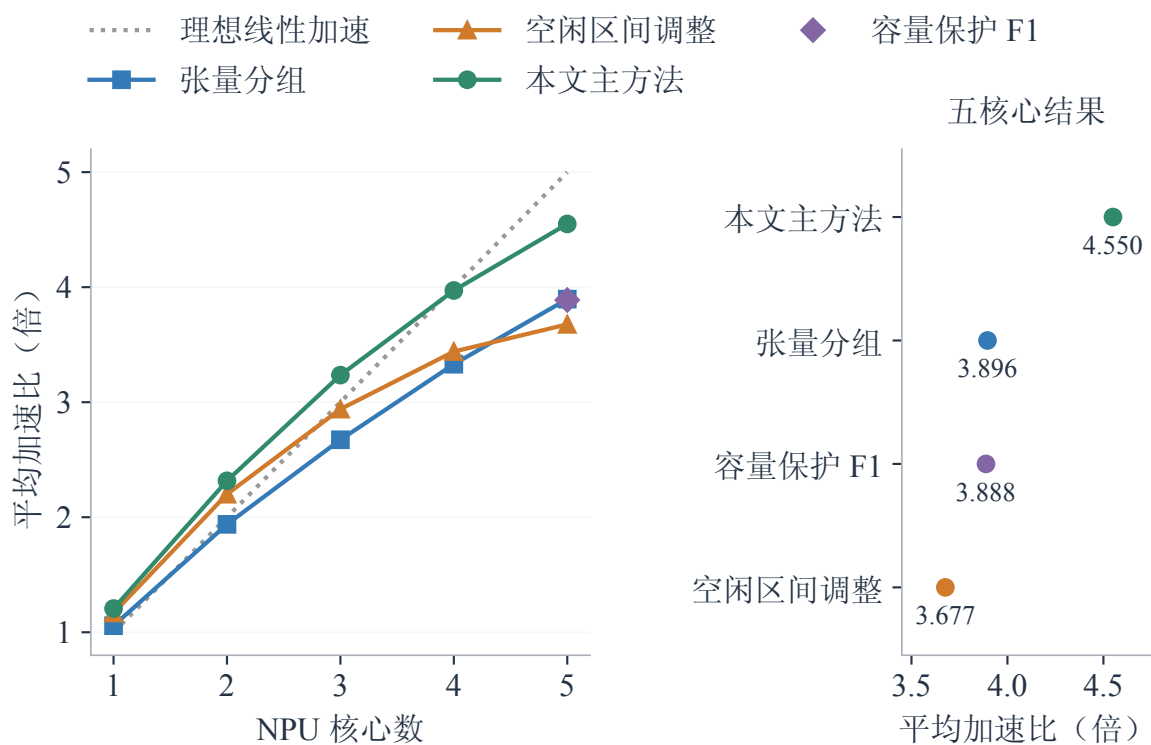


图 5-6 问题二各方法平均加速比对比

图注：问题二四种固定方法的多核平均加速比对比。左侧展示三种全覆盖方法在 1~5 核心下的逐图平均加速比曲线与容量保护 F1 方法在 5 核心下的独立测试点，灰色点线为理想线性加速参照；右侧为 5 核心场景下各方法的横向对比展开。

表 5-2 搬运量与完成时间权衡反例分析（C04）

用例名称	本文主算法 / cycles	C04 对照方案 / cycles	相对变慢幅度
case_005	33,515	38,209	+14.01%
case_016	2,091,375	2,399,125	+14.72%
case_019	16,247	28,514	+75.50%
case_025	942,938	975,658	+3.47%
case_039	449,653	494,942	+10.07%
case_075	343,930	452,120	+31.46%

时间增加了 75.50%（Makespan 由 16,247 cycles 增至 28,514 cycles）。图中各核心横道分别对应各个核心的 Task 与 Pipe 活跃区间（例如 C0:T0 与 C3:T3 分别表示 Core 0 与 Core 3 上排定的具体 Task 任务段）。在全景时序中，主算法于 16,247 cycles 即完成了全图计算，而 C04 方案持续运行至 28,514 cycles。在 13,800 至 17,400 cycles 的局部放大窗口中，C04 方案在各个核心的 Task 之间存在未记录流水线操作的间隔。需要严格说明的是，图中未着色的空白区域仅代表仿真器在此时间段内未调度对应的流水线事件，客观反映对应流水线未记录操作占用，此处不作未经微架构事件级归因分析的机理臆测。

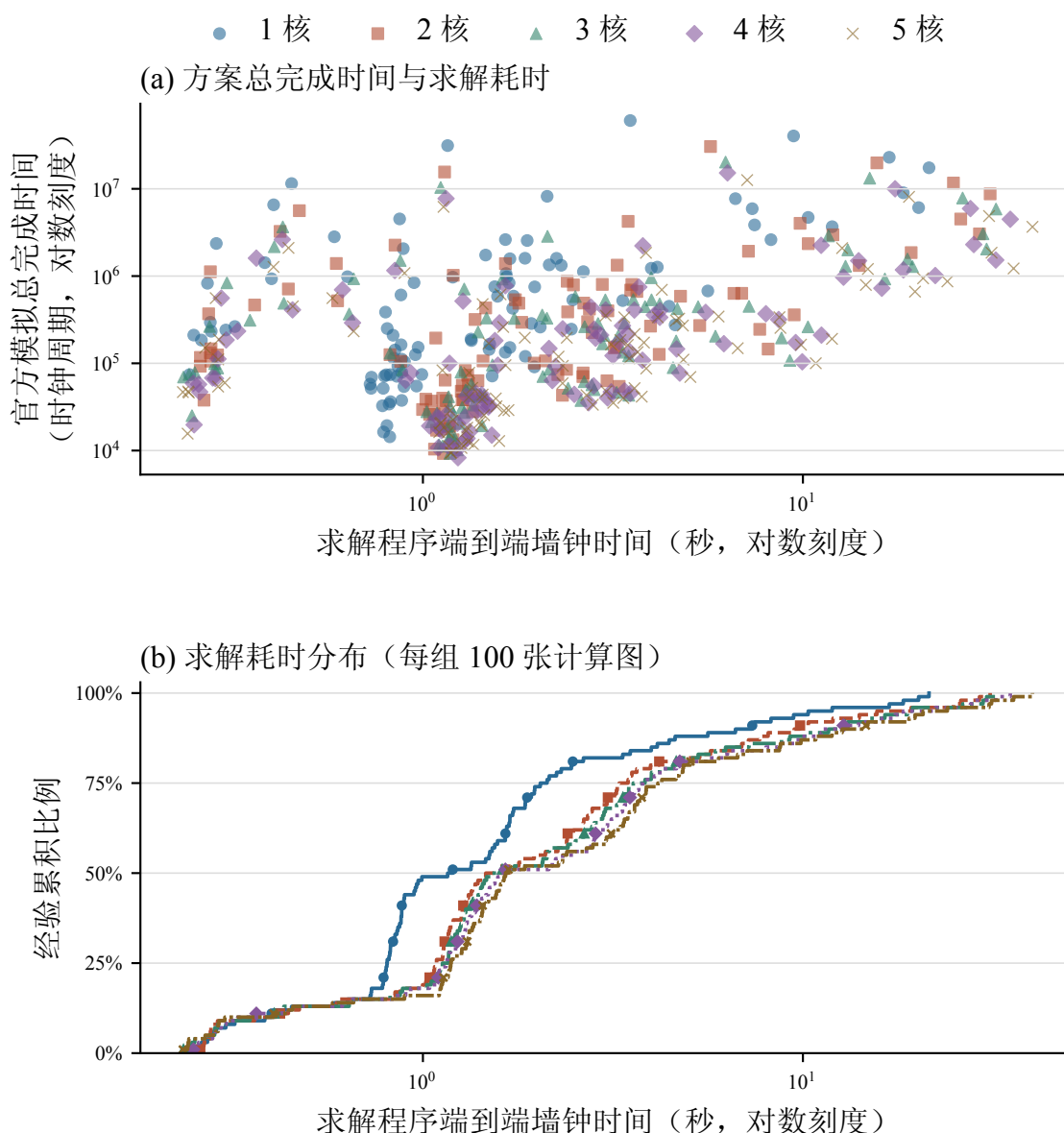


图 5-7 问题二方案完成时间与求解耗时分布

图注：问题二主算法方案质量与求解耗时分布。上图展示 500 个测试组合在官方离散事件仿真下的 **Makespan** 与求解器端到端墙钟时间双对数散点分布；下图展示各核心数下（每组 100 用例）求解耗时的经验累积分布函数（ECDF）。评测在共享主机单 worker 进程环境下完成，计时涵盖子进程启动、图解析、在线 E2 评测与方案写出全流程。

上述限定反例表明：单纯优化静态通信数据量并不能直接保证任务执行时间的缩短。二核局部超图割所度量的仅为换入换出前的原始数据复制量，且流水线工作量上限守卫亦不保证片上私有缓存完全避免溢出（spill）。

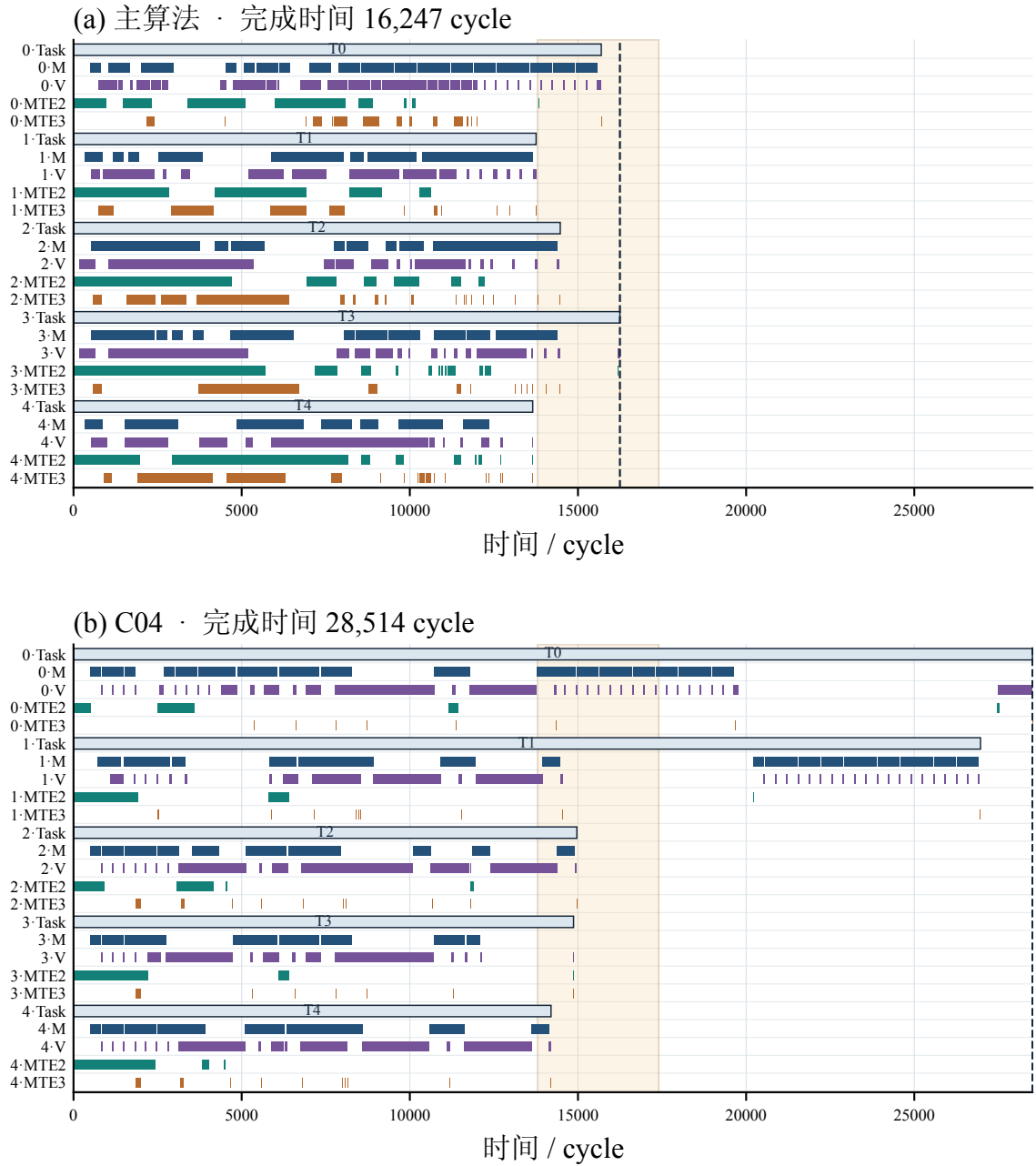


图 5-8 主算法与对照方案的完整执行时序

图注：case_019（5 核心）主算法与 C04 对照方案全景执行时序对比。展示两方案在全流程时间尺度（0 至 28,514 cycles）下各核心 Task 与 Pipe 活跃区间；C04 方案无容量不足引起的缓存换入换出（spill 为 0），但完成时间（28,514 cycles）劣于主算法（16,247 cycles）。

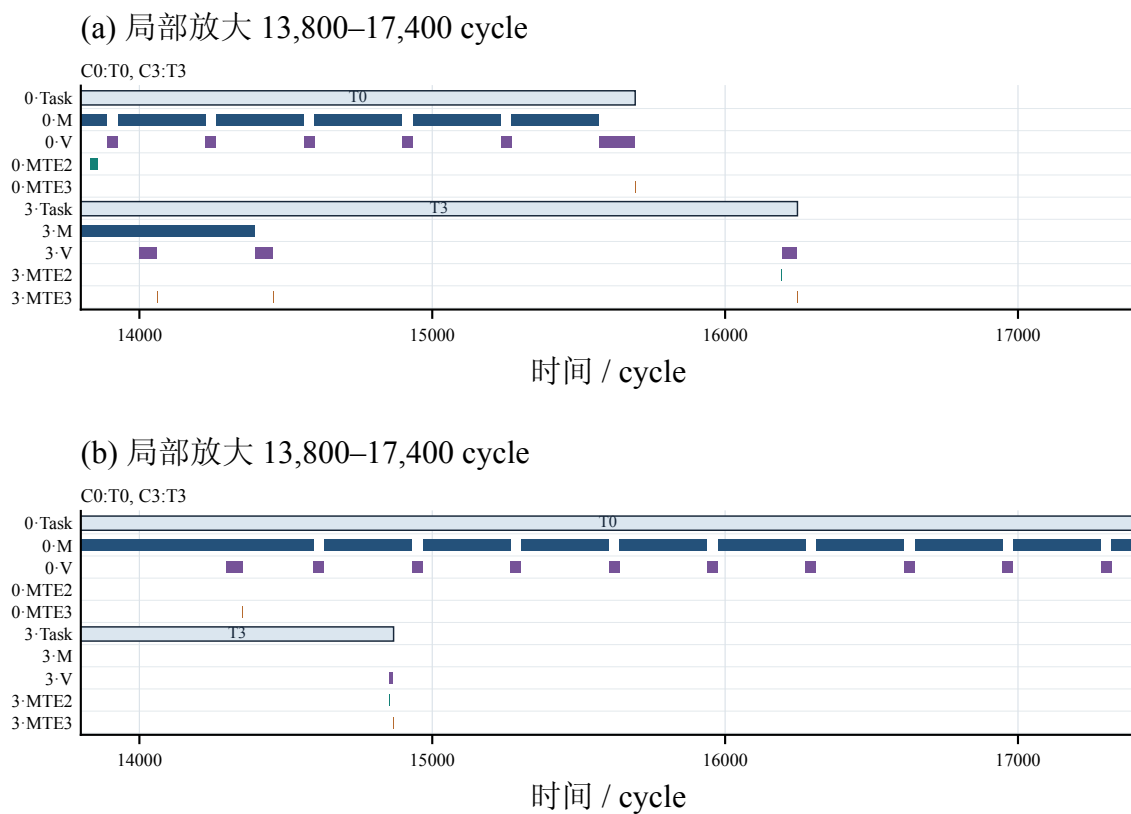


图 5-9 主算法与对照方案的局部执行时序

图注：case_019（5 核心）两方案在关键局部时间窗（13,800 至 17,400 cycles）的时序放大对比。展示主算法与 C04 在 Core 0（C0:T0）及 Core 3（C3:T3）等任务上的流水线事件分布。图中未着色空白区域仅客观反映对应流水线未记录操作占用，不作未经事件级归因分析的机理臆测。

6 问题三：共享只读缓存下的多核调度

6.1 问题分析与缓存访问约束

问题三在问题二的多核心硬件任务 (Task) 合并执行模型基础上, 引入了全部计算核心共享的片上只读缓存 (Shared Read-only Cache, 后文简称为 Cache)。该 Cache 容量固定为 1 MiB (1,048,576 字节), 只读缓存命中读取总带宽为 250 B/cycle。在这一设定下, 各个计算核心除了访问各自片上私有的 L1 Buffer 与 Unified Buffer (UB) 之外, 符合条件的 COPY_IN 搬入操作可优先向该共享 Cache 发起读取查询, 以减少访问外部主存 (DDR, 所有并发 DDR 传输共享总带宽为 60 B/cycle) 带来的时钟周期开销; 若发生缓存未命中, 则在对应数据经由 DDR 链路彻底搬运完成的时刻, 方被正式写入并插入 Cache。

依据官方评测程序的具体实现, 共享只读 Cache 的访问行为与状态迁移严格遵循如下规则:

1. **查询时机与命中判定:** 当某个核心启动数据搬入操作 COPY_IN、能够确定待搬入张量的全局逻辑标识且数据大小大于 0 字节时, 该操作在启动时刻立即向 Cache 发起查询。若 Cache 中已存在该逻辑标识对应的数据块, 则判定为**缓存命中** (cache hit), 数据直接经由 Cache 读入该核心的片上私有缓存, 数据传输速率受 Cache 带宽限制; 若 Cache 中不存在该数据, 则判定为**缓存未命中** (cache miss), 核心必须向片外 DDR 发起读取请求。
2. **数据插入与在途约束:** 当发生缓存未命中时, 数据通过 DDR 链路进行搬运, 并受系统共享总带宽 (60 B/cycle) 及并发链路仲裁约束。**数据仅在 DDR 搬运彻底完成的时刻, 方被正式写入并插入 Cache。**若某个张量正在由其他核心从 DDR 读取中 (传输尚未结束), 后继核心发起的相同张量读取请求由于 Cache 中尚无完整有效副本, 无法被判定为命中, 依然作为未命中处理;
3. **先入先出替换策略 (FIFO):** 当 Cache 当前占用总字节数与待插入张量大小之和超过容量上限 (1 MiB) 时, Cache 依据各数据块的首次插入时间先后顺序 (通过有序队列维护并按占用总字节数进行容量控制), 依次逐出最早存入的数据块, 直至释放出足够的总字节容量以容纳新张量。该共享只读缓存仅对总字节数进行总量控制, 不涉及物理地址连续性或碎片约束, 并非核心私有的 L1/UB 物理分配器。**缓存命中操作不会更新或刷新该数据块的入队时间戳, 亦不改变其在 FIFO 队列中的淘汰顺序;**
4. **超大张量不插入规则:** 对于单份数据大小严格超过 Cache 总容量 (1 MiB) 的张量, 系统直接通过 DDR 链路完成搬运, 绝不尝试将其写入 Cache, 以避免无意义地清空 Cache 中已有的其他活跃数据;
5. **硬件同步与数据依赖不变性:** 共享 Cache 的引入并未免除计算图原有的前序数据依赖约束, 亦未消除跨核心数据传递时必须满足的 500 cycles 硬件同步等待。即使后继核心从 Cache 中命中了前序算子生成的数据, 该读取操作依然必须在前序算子写出操作完成 500 cycles 之后方可启动。

上述时序规则表明, 调度算法排定的算子执行顺序直接决定了各张量读取请求在时间轴上的先后位置。由于 Cache 具有容量上限与 FIFO 逐出机制, 单纯统计静态命中次数或总命中字节数, 并不能直接反映程序完成时间 (Makespan) 的变化趋势。

根据 case_021 在 3 核心配置下的官方离散事件仿真记录, 张量 t1000000001 (数据大小为 4096 B) 的生命周期 (如表 6-1 所示) 呈现了典型的 Cache 状态迁移过程。由于 FIFO 队列仅依据首次插入时间排序, 第 4956 cycle 的命中读取并未刷新其在队列中的时间戳, 该张量最终于第 323850 cycle 因新张量插入导致容量超限而被逐出。